

Tipado simple — Resolución

tipado-simple.hs — typecheck / evalExp / fold / rec / simplify

Tipos de datos

```
data TType = Number | Boolean | Text
```

```
type Variable = String
```

```
data Exp
  = LNum Int
  | LText String
  | Var Variable
  | AritOp (Int -> Int -> Int) Exp Exp
  | RelOp (Int -> Int -> Bool) Exp Exp
  | TextAritOp (String -> String) Exp
  | TextNumOp (String -> Int) Exp
```

```
data Value = VInt Int | VBool Bool | VText String | TypeError
```

```
data Cmd
  = Skip
  | StoreInput Variable String
  | PrintVar Variable
  | Seq Cmd Cmd
  | If Exp Cmd Cmd
```

typecheck

```
typecheck :: Exp -> Maybe TType
typecheck (LNum n)      = Just Number
typecheck (LText s)     = Just Text
typecheck (Var v)       = Just Text
typecheck (AritOp fi e1 e2) = typecheckAritOp (typecheck e1) (typecheck e2)
typecheck (RelOp fib e1 e2) = typecheckRelOp (typecheck e1) (typecheck e2)
typecheck (TextAritOp fs e) = typecheckTextAritOp (typecheck e)
typecheck (TextNumOp fsi e) = typecheckTextNumOp (typecheck e)
```

```
typecheckAritOp :: Maybe TType -> Maybe TType -> Maybe TType
typecheckAritOp (Just Number) (Just Number) = Just Number
typecheckAritOp _ _ = Nothing
```

```
typecheckRelOp :: Maybe TType -> Maybe TType -> Maybe TType
typecheckRelOp (Just Number) (Just Number) = Just Boolean
typecheckRelOp _ _ = Nothing
```

```
typecheckTextAritOp :: Maybe TType -> Maybe TType
typecheckTextAritOp (Just Text) = Just Text
typecheckTextAritOp _ = Nothing
```

```
typecheckTextNumOp :: Maybe TType -> Maybe TType
typecheckTextNumOp (Just Text) = Just Number
typecheckTextNumOp _ = Nothing
```

evalExp

```
evalExp :: Exp -> (Variable -> String) -> Value
evalExp (LNum n) f      = VInt n
evalExp (LText s) f     = VText s
evalExp (Var v) f       = VText (f v)
evalExp (AritOp fi e1 e2) f = evalAritOp (evalExp e1 f) (evalExp e2 f) fi
evalExp (RelOp fib e1 e2) f = evalRelOp (evalExp e1 f) (evalExp e2 f) fib
evalExp (TextAritOp fs e) f = evalTextAritOp (evalExp e f) fs
evalExp (TextNumOp fsi e) f = evalNumOp (evalExp e f) fsi
```

```

evalAritOp :: Value -> Value -> (Int -> Int -> Int) -> Value
evalAritOp (VInt n) (VInt m) f = VInt (f n m)
evalAritOp _ _ f = TypeError

evalRelOp :: Value -> Value -> (Int -> Int -> Bool) -> Value
evalRelOp (VInt n) (VInt m) f = VBool (f n m)
evalRelOp _ _ f = TypeError

evalTextAritOp :: Value -> (String -> String) -> Value
evalTextAritOp (VText s) f = VText (f s)
evalTextAritOp _ f = TypeError

evalNumOp :: Value -> (String -> Int) -> Value
evalNumOp (VText s) f = VInt (f s)
evalNumOp _ f = TypeError

```

tvalue

```

tvalue :: Value -> Maybe TType
tvalue (VInt _) = Just Number
tvalue (VBool _) = Just Boolean
tvalue (VText _) = Just Text
tvalue TypeError = Nothing

```

Demostración: typecheck = tvalue . flip evalExp env

Sea `env :: Variable -> String` una función cualquiera, fija y arbitraria (dado por el "para todo `env`" del enunciado).

Por principio de extensionalidad, siendo `exp` una `Exp` cualquiera, quiero demostrar que:

¿ para toda `exp`. `typecheck exp = (tvalue . flip evalExp env) exp` ?

por def. de compose, es equivalente decir:

¿ para toda `exp`. `typecheck exp = tvalue (flip evalExp env exp)` ?

por principio de inducción estructural sobre `exp`

Planteo

casoBase1) siendo `n` un número cualquiera. `exp = LNum n`
 ¿ `typecheck (LNum n) = tvalue (flip evalExp env (LNum n))` ?

casoBase2) siendo `s` una string cualquiera. `exp = LText s`
 ¿ `typecheck (LText s) = tvalue (flip evalExp env (LText s))` ?

casoBase3) siendo `v` una variable cualquiera. `exp = Var v`
 ¿ `typecheck (Var v) = tvalue (flip evalExp env (Var v))` ?

casoInductivo1) siendo `fi :: (Int -> Int -> Int)` cualquiera, `e1, e2` unas expresiones cualquiera. `exp = AritOp fi e1 e2`
 HI1) ¿ `typecheck e1 = tvalue (flip evalExp env e1)` !
 HI2) ¿ `typecheck e2 = tvalue (flip evalExp env e2)` !
 TI) ¿ `typecheck (AritOp fi e1 e2) = tvalue (flip evalExp env (AritOp fi e1 e2))` ?

casoInductivo2) siendo `fib :: (Int -> Int -> Bool)` cualquiera, `e1, e2` unas expresiones cualquiera. `exp = RelOp fib e1 e2`
 HI1) ¿ `typecheck e1 = tvalue (flip evalExp env e1)` !
 HI2) ¿ `typecheck e2 = tvalue (flip evalExp env e2)` !
 TI) ¿ `typecheck (RelOp fib e1 e2) = tvalue (flip evalExp env (RelOp fib e1 e2))` ?

casoInductivo3) siendo `fs :: (String -> String)` cualquiera, `e` una expresión cualquiera. `exp = TextAritOp fs e`
 HI) ¿ `typecheck e = tvalue (flip evalExp env e)` !
 TI) ¿ `typecheck (TextAritOp fs e) = tvalue (flip evalExp env (TextAritOp fs e))` ?

casoInductivo4) siendo `fsi :: (String -> Int)` cualquiera, `e` una expresión cualquiera. `exp = TextNumOp fsi e`
 HI) ¿ `typecheck e = tvalue (flip evalExp env e)` !
 TI) ¿ `typecheck (TextNumOp fsi e) = tvalue (flip evalExp env (TextNumOp fsi e))` ?

Demuestro

casoBase3)

LI)

```
typecheck (Var v)
=
typecheck.3
Just Text
```

LD)

```
tvalue (flip evalExp env (Var v))
=
flip
tvalue (evalExp (Var v) env)
=
evalExp.3
tvalue (VText (f v))
=
tvalue.3
Just Text
```

Para este caso vale.

casoInductivo2)

LI)

```
typecheck (RelOp fib e1 e2)
=
typecheck.5
typecheckRelOp (typecheck e1) (typecheck e2)
=
HI1 y HI2
typecheckRelOp (tvalue (evalExp e1 env)) (tvalue (evalExp e2 env))
=
lema-typeRelOp
tvalue (evalRelOp (evalExp e1 env) (evalExp e2 env) fib)
```

LD)

```
tvalue (flip evalExp env (RelOp fib e1 e2))
=
flip
tvalue (evalExp (RelOp fib e1 e2) env)
=
evalExp.5
tvalue (evalRelOp (evalExp e1 env) (evalExp e2 env) fib)
```

Vale para este caso.

lema-typeRelOp

Sea $v1 :: \text{Value}$, $v2 :: \text{Value}$ cualquiera, quiero ver que:

¿ para todo $v1, v2$. $\text{typecheckRelOp} (\text{tvalue } v1) (\text{tvalue } v2) = \text{tvalue} (\text{evalRelOp } v1 \ v2 \ \text{fib})$?

por analisis de casos sobre $v1, v2$

c1) siendo n, m unos números cualquiera. $v1 = \text{VInt } n$, $v2 = \text{VInt } m$

¿ $\text{typecheckRelOp} (\text{tvalue } (\text{VInt } n)) (\text{tvalue } (\text{VInt } m)) = \text{tvalue} (\text{evalRelOp } (\text{VInt } n) (\text{VInt } m) \ \text{fib})$?

c2) $v1 \neq \text{VInt } n$ ó $v2 \neq \text{VInt } m$

¿ $\text{typecheckRelOp} (\text{tvalue } v1) (\text{tvalue } v2) = \text{tvalue} (\text{evalRelOp } v1 \ v2 \ \text{fib})$?

c1)

LI)

```
typecheckRelOp (tvalue (VInt n)) (tvalue (VInt m))
=
tvalue.1
typecheckRelOp (Just Number) (Just Number)
=
typecheckRelOp.1
Just Boolean
```

LD)

```
tvalue (evalRelOp (VInt n) (VInt m) fib)
=
evalRelOp.1
tvalue (VBool (fib n m))
=
tvalue.2
Just Boolean
```

Vale para este caso.

```

c2)
LI)
  typecheckRelOp (tvalue (VBool b1)) (tvalue (VBool b2))
=
  typecheckRelOp Nothing Nothing
=
  Nothing
LD)
  tvalue (evalRelOp v1 v2 fib)
=
  tvalue (TypeError)
=
  Nothing

```

Vale para este caso.
La propiedad vale.

Punto 4 — foldExp / recExp

```

foldExp
  :: (Int -> b) -> (String -> b) -> (Variable -> b)
  -> ((Int -> Int -> Int) -> b -> b -> b)
  -> ((Int -> Int -> Bool) -> b -> b -> b)
  -> ((String -> String) -> b -> b)
  -> ((String -> Int) -> b -> b)
  -> Exp -> b
foldExp n t v a r ta tn (LNum x)          = n x
foldExp n t v a r ta tn (LText s)         = t s
foldExp n t v a r ta tn (Var var)         = v var
foldExp n t v a r ta tn (AritOp fi e1 e2) =
  a fi (foldExp n t v a r ta tn e1) (foldExp n t v a r ta tn e2)
foldExp n t v a r ta tn (RelOp fib e1 e2) =
  r fib (foldExp n t v a r ta tn e1) (foldExp n t v a r ta tn e2)
foldExp n t v a r ta tn (TextAritOp fs e) = ta fs (foldExp n t v a r ta tn e)
foldExp n t v a r ta tn (TextNumOp fsi e) = tn fsi (foldExp n t v a r ta tn e)

recExp
  :: (Int -> b) -> (String -> b) -> (Variable -> b)
  -> ((Int -> Int -> Int) -> Exp -> b -> Exp -> b -> b)
  -> ((Int -> Int -> Bool) -> Exp -> b -> Exp -> b -> b)
  -> ((String -> String) -> Exp -> b -> b)
  -> ((String -> Int) -> Exp -> b -> b)
  -> Exp -> b
recExp n t v a r ta tn (LNum x)          = n x
recExp n t v a r ta tn (LText s)         = t s
recExp n t v a r ta tn (Var var)         = v var
recExp n t v a r ta tn (AritOp fi e1 e2) =
  a fi e1 (recExp n t v a r ta tn e1) e2 (recExp n t v a r ta tn e2)
recExp n t v a r ta tn (RelOp fib e1 e2) =
  r fib e1 (recExp n t v a r ta tn e1) e2 (recExp n t v a r ta tn e2)
recExp n t v a r ta tn (TextAritOp fs e) = ta fs e (recExp n t v a r ta tn e)
recExp n t v a r ta tn (TextNumOp fsi e) = tn fsi e (recExp n t v a r ta tn e)

```

Punto 5 — foldCmd / recCmd / simplify

```

foldCmd
  :: b -> (Variable -> String -> b) -> (Variable -> b)
  -> (b -> b -> b) -> (Exp -> b -> b -> b)
  -> Cmd -> b
foldCmd s st pr sq i Skip                = s
foldCmd s st pr sq i (StoreInput var str) = st var str
foldCmd s st pr sq i (PrintVar var)       = pr var
foldCmd s st pr sq i (Seq c1 c2)         =
  sq (foldCmd s st pr sq i c1) (foldCmd s st pr sq i c2)
foldCmd s st pr sq i (If e c1 c2)        =
  i e (foldCmd s st pr sq i c1) (foldCmd s st pr sq i c2)

```

```

recCmd
  :: b -> (Variable -> String -> b) -> (Variable -> b)
  -> (Cmd -> b -> Cmd -> b -> b) -> (Exp -> Cmd -> b -> Cmd -> b -> b)
  -> Cmd -> b
recCmd s st pr sq i Skip                = s
recCmd s st pr sq i (StoreInput var str) = st var str
recCmd s st pr sq i (PrintVar var)       = pr var
recCmd s st pr sq i (Seq c1 c2)          =
  sq c1 (recCmd s st pr sq i c1) c2 (recCmd s st pr sq i c2)
recCmd s st pr sq i (If e c1 c2)         =
  i e c1 (recCmd s st pr sq i c1) c2 (recCmd s st pr sq i c2)

```

explicita

```

simplifyCmd :: Cmd -> Cmd
simplifyCmd Skip                = Skip
simplifyCmd (StoreInput var s) = StoreInput var s
simplifyCmd (PrintVar var)     = PrintVar var
simplifyCmd (Seq c1 c2)        = simplifySeq (simplifyCmd c1) (simplifyCmd c2)
simplifyCmd (If e c1 c2)       = simplifyIf e (simplifyCmd c1) (simplifyCmd c2)

```

```

simplifySeq :: Cmd -> Cmd -> Cmd
simplifySeq Skip c2 = c2
simplifySeq c1 Skip = c1
simplifySeq c1 c2   = Seq c1 c2

```

```

simplifyIf :: Exp -> Cmd -> Cmd -> Cmd
simplifyIf (RelOp fib (LNum n) (LNum m)) c1 c2 = if fib n m then c1 else c2
simplifyIf e c1 c2                             = If e c1 c2

```

implicita

```

simplifyCmd' :: Cmd -> Cmd
simplifyCmd' = foldCmd Skip StoreInput PrintVar simplifySeq simplifyIf

```